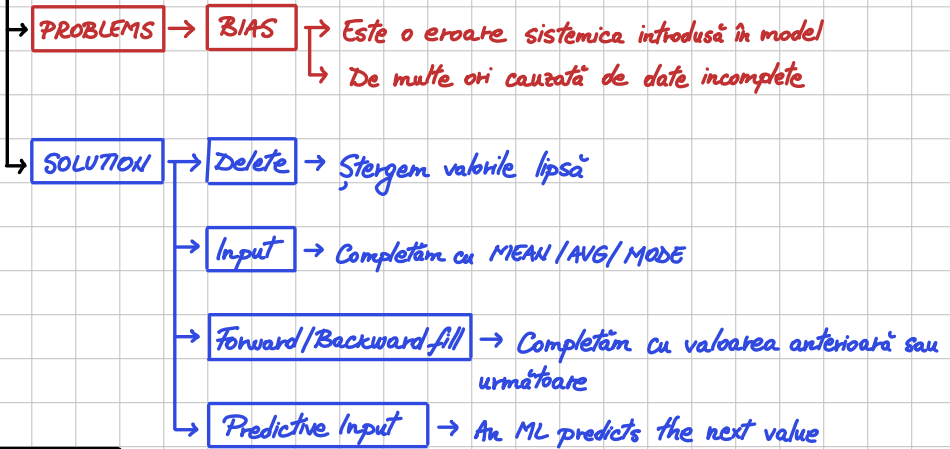


= Data Preprocessing & Cleaning =

Data Quality

- ① COMPLETENESS
- ② ACURACY
- ③ CONSISTENCY
- ④ TIMEINESS

① COMPLETENESS



② ACURACY



③ CONSISTENCY



DATA PROCESSING

- ① MCAR → Missing Completely At Random → lipsesc aleator, fără legătură cu alte variabile
- ② MAR → Missing At Random → lipsesc, dar probabilitatea lipsei depinde de alte variabile observabile
- ③ MNAR → Missing Not At Random → lipsesc dintr-un motiv intern, legat de valoarea în sine

SCALAREA VARIABELOR

- **PROBLEMS** → Diferite variabile pot avea scale diferite (ex: salariu vs. vârstă)
- **SOLUTION** → Punem toate variabilele în aceeași scală de valori

METODE DE SCALARE

① **SCALARE MIN-MAX** → **FORMULA** → $(x - \text{MIN}) / (\text{MAX} - \text{MIN})$

- Păstrează proporții relative
- Foarte sensibil la outliers

② **SCALARE STANDARD** → **FORMULA** → $(x - \text{MEDIE}) / \text{DEVIATIA STANDARD}$

- Valorile sunt centrate și măsurate în "câte deviații standard" sunt de la medie
- Mai robust la outliers decât Min-Max
- Nu garantează valori între 0 și 1

MIN-MAX vs. STANDARD SCALE

EXEMPLU : Preț bilet în setul de date Titanic = 7.25

- Min-Max = 0.014 → foarte mic față de maxim
- StandardScaler = -0.5 → puțin sub medie (media e 0)

DIFERENȚA :

- Min-Max compară față de minimul și maximul posibil
- StandardScaler compară față de medie și variație

CATEGORICAL DATA → Valori precum "male", "female"

TRANSFORMATION

LABEL ENCODING

→ Categorie = număr → $\begin{cases} \text{male} = 1 \\ \text{female} = 0 \end{cases}$

ONE-HOT ENCODING

→ Coloană separată pentru fiecare categorie
→ Coloana ia valori 0 și 1

Coloana Embark ia valorile {C, Q, S}

→ One-Hot Encoding → $\begin{cases} \text{Embark_C} \rightarrow 1 \text{ dacă e C, altfel } 0 \\ \text{Embark_Q} \rightarrow 1 \text{ dacă e Q, altfel } 0 \\ \text{Embark_S} \rightarrow 1 \text{ dacă e S, altfel } 0 \end{cases}$

LABEL ENCODING VS. ONE-HOT ENCODING

LABEL ENCODING

→ Simplu, dar introduce ordine false
→ Bun doar pentru categorii ordinale (ex: low < med < high)

ONE-HOT ENCODING

→ Evită ordinea, dar creează mai multe coloane
→ Preferat pentru categorii nominale (Sex, Embarked)

= EXPLORATORY DATA ANALYSIS =

CORELATIE

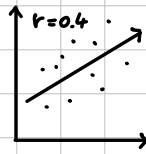
①

PEARSON →

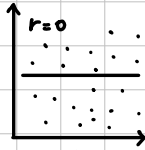
$$r = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum (x_i - \bar{x})^2 \cdot \sum (y_i - \bar{y})^2}}$$

→ Cea mai comună

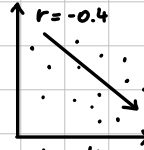
→ Măsoară relația liniară între 2 variabile



Positive
Correlation



No
Correlation



Negative
Correlation

②

SPEARMAN →

$$\rho = 1 - \frac{6 \sum d_i^2}{n(n^2 - 1)}$$

→ $d = R(x_i) - R(y_i)$
difference between ranks

→ n - nr. of observations

$$\rho = \frac{\text{Cov}(R_x, R_y)}{\sigma_{R_x} \sigma_{R_y}}$$

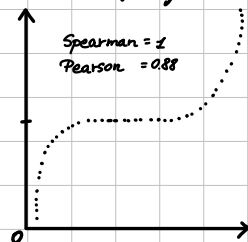
→ R_x, R_y ranks of X, Y
→ σ - standard deviation

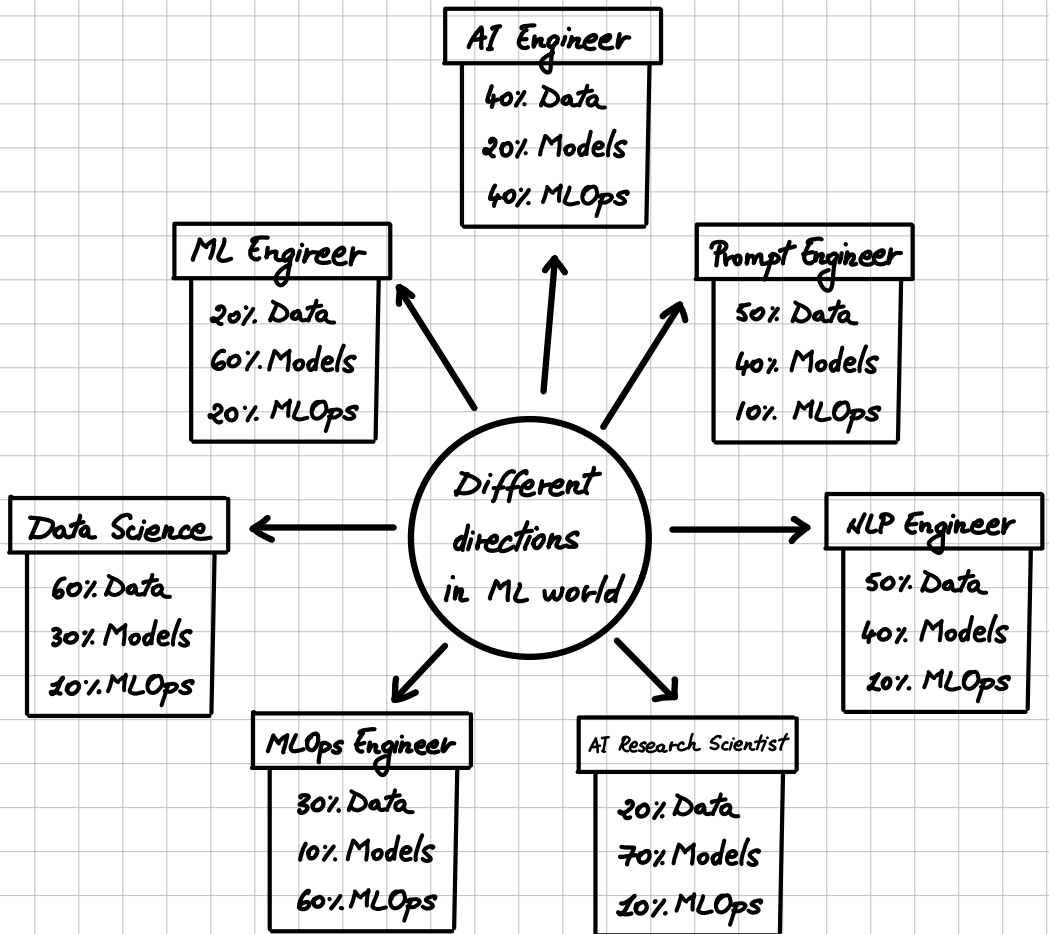
→ Măsoară relația monotonică între 2 variabile → Cum crește / scade X în funcție de Y

→ Nu trebuie să fie direct proporționale

→ Este utilă dacă:

- Datele NU sunt distribuite normal
- Avem outliers





= LINEAR REGRESSION =

LINEAR REGRESSION → Scop → Prețicem o valoare continuă (ex: notă, preț)

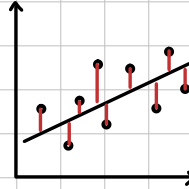
→ INTUITIE → Multe ore de studiu (X) → de regulă note mai mari (Y)

→ MODEL → O linie care rezumă relația dintre X și Y

FORMULĂ

ECUAȚIA DREPTEI

$$y = mx + n$$



→ n → INTERCEPT → Valoarea lui Y când X=0 (nota de start)

→ m → PANTA → Cât se schimbă Y atunci când X crește cu o unitate

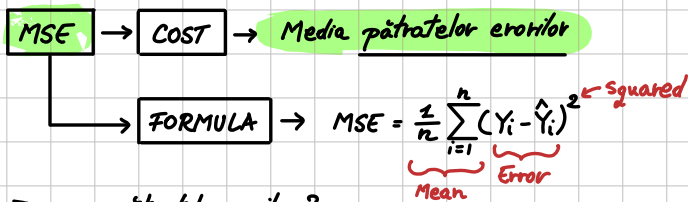
→ INTUITIE → PANTA → dă "rata de schimbare"

→ INTERCEPT → dă "nivelul de bază"

EROARE → REZIDUU → Diferența dintre predicție și valoarea reală

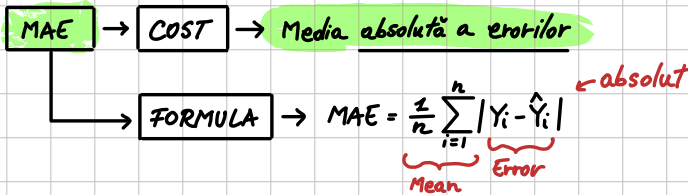
COST → MSE → Media pătratelor erorilor

→ SCOP → Să minimizăm costul (erorile totale)



De ce pătratul erorilor?

- Fără pătrat erorile pozitive și negative s-ar anula
- Outlierii trebuie să "doară mai mult" → pătratul îi face vizibil
- MSE penalizează exponențial erorile mari
- MAE (media absolută) e mai robustă



De ce valoarea absolută a erorilor?

- Fără valoarea absolută, erorile pozitive și negative s-ar anula
- MAE este robust la outlierii → nu exagerază erorile
- Folosește distanța L1 → Măsoară "cât de departe" e predicția în medie
- Oferă o măsură intuitivă: eroarea medie în unitate originală a problemei

OLS → ORDINARY LEAST SQUARE

→ FORMULA → $E(m,n) = \sum (y_i - (mx_i + n))^2$

- Ca să găsim cel mai bun line, vrem să minimizăm funcția de cost cât de mult posibil

Exemplu: Vrem să găsim linia $y = mx + n$ care trece cât mai aproape de toate punctele. Avem 3 puncte:

x	y
1	2
2	3
3	5

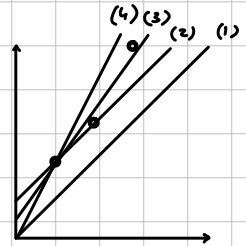
m	n	$\hat{y}$	Total Error $E = (y - \hat{y})^2$
1	0	[1, 2, 3]	$(2-1)^2 + (3-2)^2 + (5-3)^2 = 6$
1	1	[2, 3, 4]	$(2-2)^2 + (3-3)^2 + (5-4)^2 = 1$
2	0	[2, 3, 4]	$(2-2)^2 + (3-4)^2 + (5-6)^2 = 2$
1.5	0.5	[2.0, 3.5, 5.0]	$(2-2)^2 + (3-3.5)^2 + (5-5)^2 = 0.25$

→ $y = x$ (1)

→ $y = x + 1$ (2)

→ $y = 2x$ (3)

→ $y = 1.5x + 0.5$ (4)



- În loc să folosim brute force pentru găsirea valorii potrivite pentru funcția de cost, putem folosi formula OLS:

$$\hat{m} = \frac{\sum_{i=1}^N (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^N (x_i - \bar{x})^2}$$

$$\hat{n} = \bar{y} - \hat{m}\bar{x}$$

Intuiția derivatelor

→ Funcția de cost → $E(m,n) = \sum_{i=1}^N (y_i - (mx_i + n))^2$

→ Derivata parțială după m → $m = \frac{\bar{x}\bar{y} - \bar{x}\bar{y}}{\bar{x}^2 - \bar{x}^2} = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2}$

→ Derivata parțială după n → $\bar{n} = \bar{y} - m\bar{x}$

OLS



ORDINARY LEAST SQUARE

→ FORMULA

$$E(m,n) = \sum (y_i - (mx_i + n))^2$$

→ Soluție "dintr-o singură lovitură" - formula analitică a liniei cu MSE min

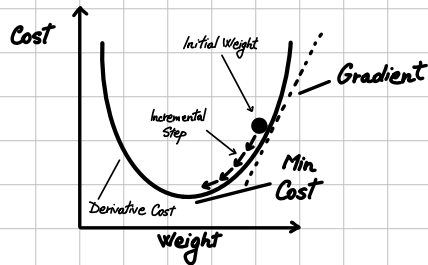
GRADIENT DESCENT

→ "Coborâre panta costului" cu pași mici până ajungem la min

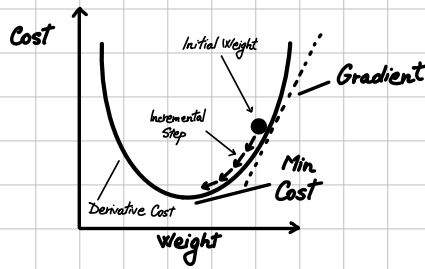
În practică:

- OLS e rapid pe seturi mici
- Gd e flexibil pentru seturi mari / modele mai complicate

$$E(m,n) = \frac{1}{2m} \sum_{i=1}^m ((mx_i + n) - y_i)^2$$



$$E(m,n) = \frac{1}{2m} \sum_{i=1}^m ((mx_i + n) - y_i)^2$$



= DERIVATA =

DERIVATA → Panta tangentei în acel punct

TANGENTA → O aproximare locală, liniară, a funcției
→ Arată în ce direcție merge funcția în imediata vecinătate a punctului

Derivata unei funcții într-un punct este panta tangentei în acel punct, iar această pantă descrie rata și direcția de variație a funcției în acel punct

- panta > 0 → funcția crește local
- panta < 0 → funcția scade local
- panta $= 0$ → funcția are un orizont (posibil vârf sau vale)

Exemplu:

$$f(x) = x^2$$

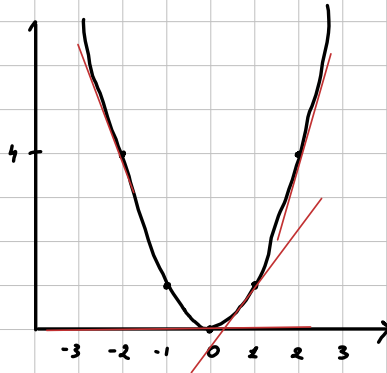
$$f'(x) = 2x$$

$$x=0 \Rightarrow f'(0)=0$$

$$x=1 \Rightarrow f'(1)=2$$

$$x=3 \Rightarrow f'(3)=6$$

$$x=-2 \Rightarrow f'(-2)=-4$$



Cum învățăm un model?

Gradientul pentru funcția de cost

- Gradient = "derivată generalizată" pentru funcții cu mai mulți parametri
- Intuiție: în 2D, gradientul arată direcția pt cea mai rapidă urcare
- Pentru scăderea costului: mergem în direcția opusă gradientului

GRADIENT DESCENT

→ pași algoritmului pentru regresie liniară

- PAS 1 → Alege o linie random (coeficienți aleatori)
- PAS 2 → Calculează gradientul (în ce direcție crește costul)
- PAS 3 → Mergi un pic în direcția opusă gradientului (scădere)
- PAS 4 → Repetă → Linia se apropie de cea optimă
- REZULTATUL → Ajungem în minimul funcției de cost (linia "cea mai bună")

Exemplu: Set de date:

x	y
1	2
2	3
3	5

Formula Cost:

$$E(m, n) = \frac{1}{2m} \sum_{i=1}^m ((mx_i + n) - y_i)^2$$

→ Doar această parte o derivăm în procesul de actualizare (m, n)

→ Nu derivăm acest termen

STEP 1: Alegem valori initiale pentru m, n

STEP 2: Calculăm gradientul pentru m

DERIVATA FAȚĂ DE m

① Derivăm termenul $(mx_i + n - y_i)^2$ față de m:

$$\frac{\partial}{\partial n} ((mx_i + n - y_i)^2) = 2(mx_i + n - y_i) \cdot x_i$$

② Împachetăm în derivata totală:

$$\frac{\partial E}{\partial n} = \frac{1}{2m} \sum_{i=1}^m 2(mx_i + n - y_i) \cdot x_i$$

③ Simplificăm:

$$\frac{\partial E}{\partial n} = \frac{1}{m} \sum_{i=1}^m (mx_i + n - y_i) x_i$$

STEP 2: Calculăm gradientul pentru n

DERIVATA FAȚĂ DE n

① Derivăm termenul $(mx_i + n - y_i)^2$ față de n:

$$\frac{\partial}{\partial n} ((mx_i + n - y_i)^2) = 2(mx_i + n - y_i)$$

② Împachetăm în derivata totală:

$$\frac{\partial E}{\partial n} = \frac{1}{2m} \sum_{i=1}^m 2(mx_i + n - y_i)$$

③ Simplificăm:

$$\frac{\partial E}{\partial n} = \frac{1}{m} \sum_{i=1}^m (mx_i + n - y_i)$$

STEP 3: Scădem gradientul din m, n

Actualizarea lui m :

$$m_{\text{nou}} = m - \mathcal{L} \cdot \left(\frac{1}{N} \sum_{i=1}^N ((m \cdot x_i + n) - y_i) \cdot x_i \right)$$

→ Scădem controlat din valoarea calculată

Actualizarea lui n :

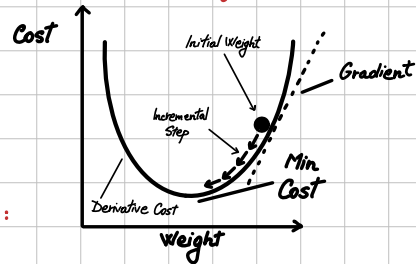
$$n_{\text{nou}} = n - \mathcal{L} \cdot \left(\frac{1}{N} \sum_{i=1}^N ((m \cdot x_i + n) - y_i) \right)$$

→ LEARNING RATE → controlează mărimea pasului

Ce rol are learning rate?

- $\mathcal{L}$ → prea mare → sari peste min
- $\mathcal{L}$ → prea mic → înveți lent
- $\mathcal{L}$ → optim → cobori eficient spre minimul funcției de cost

cu care actualizăm parametri la fiecare iterație



Metaforă:

"Este mărimea pașilor atunci când cobori un munte:

nici prea mari că te împiedici

nici prea mici că faci o zi întreagă"

STEP 4: Repetăm până ajungem la valoarea optimă pentru m, n

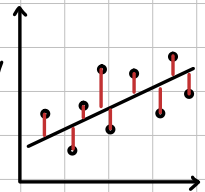
LEARNING RATE → Pasul de învățare

- Prea mare → sărim peste minim (oscilații, instabil)
- Prea mică → ajungem la minim, dar extrem de lent
- Scop: Un pas suficient de mare ca să avansăm, dar suficient de mic ca să fie stabil



ANALIZA ERORILOR

- Linia nu e totul → Vrem să știm unde și cum greșește modelul
- Reziduu → Diferența dintre valoarea reală și cea prezisă
- Ne arată ce parte din realitate nu a fost surprinsă
- Model bun → reziduurile sunt aleatoare, fără tipar
- Model prost → reziduurile formează modele clare (curbe, pâlnie)



GRADIENT DESCEN → Nu garantează mereu soluția globală, decât în funcții convexe)

- De unde știm când am ajuns la soluția optimă?

② Gradientul este aproape 0 → $|\nabla J(\theta)| \approx 0$, unde $\theta = \begin{bmatrix} \theta_1 \\ \vdots \\ \theta_m \end{bmatrix}$, $J(\theta) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$,
 $h_{\theta}(x) = \theta^T x$ pred. modelului

② Costul/Loss nu mai scade → $|J_t - J_{t-1}| < f \approx 10^{-6} \vee 10^{-7}$, $t \rightarrow \text{epoch}$

③ Actualizarea parametrilor devine foarte mică: $\begin{cases} |m_t - m_{t-1}| < f \\ |n_t - n_{t-1}| < f \end{cases}$

④ După un anumit număr de epoci

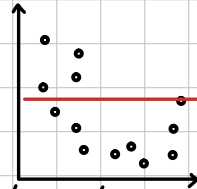
VANISHING GRADIENT PROBLEM → Gradientul care devine aproape 0 în rețele adânci

→ Modelul nu învață pentru că derivata se micșorează la fiecare strat

→ Apare din cauza activării sigmoid/tanh și inițializării slabe

Reziduurile sănătoase vs. "curba"

- Sănătos → Reziduuri ca un nor aleator în jurul lui 0
- Curba → Reziduuri în formă de U sau arc → Modelul linia ratează o relație neliniară
- Intuiție → Linia e prea simplă pentru o relație curbă

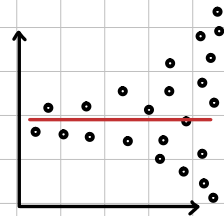


Reziduurile "pâlnie" = heteroscedasticitate

- Ipoteză → erorile au varianță constantă → Homoscedasticitate
- Heteroscedasticitate → erorile devin mai mari pentru valori mai mari
- Intuiție → Modelul
- Exemplu real:
 - Salarii → pentru joburi entry-level toți câștigă similar (reziduuri mici)
 - Pentru CEO unii câștigă 10k€, alții 50k€ (reziduuri foarte mari)

Ipoteza modelului liniar:

- Liniaritate
- Erori distribuite normal (aprox.)
- Varianță constantă (homoscedasticitate)
- Independență:
 - Salarii → pentru joburi entry-level toți câștigă similar (reziduuri mici)
 - Pentru CEO unii câștigă 10k€, alții 50k€ (reziduuri foarte mari)



= Multiple & Polynomial Regression =

MULTIPLE REGRESSION

→ O versiune mai puternică a regresiei liniare
 → Putem prezice un rezultat folosind mai multe variabile (features), nu doar una

LINEAR REGRESSION

INPUT → X

OUTPUT → Y

MULTIPLE REGRESSION

INPUT → $X_i, i \in \overline{1, n}$

OUTPUT → Y

Exemplu:

Size (ft ²)	Bedrooms	Floors	Home Age (years)	Price (thosands \$)
2104	5	1	45	460
1416	3	2	40	232
1536	3	2	30	315
852	2	1	36	178

$n = 4 \rightarrow$ nr. of features

x_1 : Size (ft²)

x_2 : Bedrooms

x_3 : Floors

x_4 : Home Age (years)

y : Price (thosands \$)

$m = 47 \rightarrow$ nr de exemple de antrenament (rows)

$$X = \begin{bmatrix} 2104 & 5 & 1 & 45 \\ 1416 & 3 & 2 & 40 \\ 1534 & 3 & 2 & 30 \\ 852 & 2 & 1 & 36 \end{bmatrix} \quad y = \begin{bmatrix} 460 \\ 232 \\ 315 \\ 178 \end{bmatrix}$$

$$\rightarrow \mathbf{x}^{(0)} = \begin{bmatrix} 2104 \\ 5 \\ 1 \\ 40 \end{bmatrix} \quad \dots \quad \mathbf{x}^{(0)} = \begin{bmatrix} 852 \\ 2 \\ 1 \\ 36 \end{bmatrix}$$

Regresie Simplă:

$$f(x) = mx + n$$

Cu mai multe variabile:

$$h(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_n x_n$$

$$h(x) = \Theta^T X$$

Exemplu:

$$h(x) = 2 + 0.5x_1 + x_2 - 3x_3 + 2.5x_4$$

GRADIENT DESCENT for multi-vars

Funcția de cost (MSE):

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

Unde:

- $J(\theta)$ → valoarea funcției de cost pentru parametrii θ
- m → numărul de exemple de antrenament
- $h_{\theta}(x^{(i)})$ → predicția modelului pentru exemplul i
- $y^{(i)}$ → valoarea reală (target) pentru exemplul i

Update rule pentru fiecare θ_j (Gradient Descent rule pentru fiecare iterație):

$$\theta_j := \theta_j - \mathcal{L} \cdot \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) \cdot x_j^{(i)}$$

Unde:

- θ_j → parametrul j care se actualizează
- $\mathcal{L}$ → rata de învățare (learning rate)
- m → numărul de exemple de antrenament
- $h_{\theta}(x^{(i)})$ → predicția modelului pentru exemplul i
- $y^{(i)}$ → valoarea reală pentru exemplul i
- $x_j^{(i)}$ → valoarea caracteristicii j pentru exemplul i

Exemplu: Pentru un singur feature:

- Cazul simplu (θ_0, θ_1) $\rightarrow y = \theta_0 + \theta_1 x$
 - Actualizăm separat:

$$\begin{cases} \theta_0 := \theta_0 - \mathcal{L} \frac{1}{m} \sum_{i=1}^m (h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)}) \\ \theta_1 := \theta_1 - \mathcal{L} \frac{1}{m} \sum_{i=1}^m (h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)}) \cdot x^{(i)} \end{cases}$$

- Pentru mai multe features:
 - La fel, dar pentru fiecare θ_j în parte
 - Gradient Descent actualizează toate simultan la fiecare iterație
 - Ideea e aceeași, doar dimensiunea crește

$$\begin{cases} \theta_0 := \theta_0 - \mathcal{L} \frac{1}{m} \sum_{i=1}^m (h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)}) \\ \theta_j := \theta_j - \mathcal{L} \frac{1}{m} \sum_{i=1}^m (h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)}) \cdot x_j^{(i)}, \quad \forall j = 1, k \end{cases}$$

- Asigură-te că toate features sunt pe aceeași scară

Exemplu:

$x_1 = \text{suprafață (0-2000)}$

$x_2 = \text{nr. dormitoare (1-5)}$

Fără scaling, contururile costului devin eliptice $\rightarrow$ GD lent

UNSCALED FEATURES

$x_1 = \text{suprafață (0-2000)}$

$x_2 = \text{nr. dormitoare (1-5)}$

- Contururile funcției de cost sunt alungite și eliptice, ceea ce face ca GD să fie lent și să oscileze ineficient
- Traectoria roșie spre minim este lungă și sinusoidală (cu multe viraje, adică multe minime locale)

VS.

SCALED FEATURES

$x_1 = \text{size} / 20$

$x_2 = \text{bedrooms} / 5$

- Contururile devin aproape circulare, ceea ce permite GD să convergă rapid și eficient
- Traectoria roșie este scurtă și directă spre

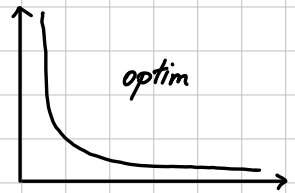
mini

LEARNING RATE

→ Controlează mărimea pașilor GD

prea mare → sare peste minim
prea mic → învață lent

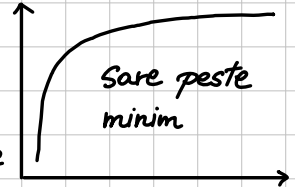
→ Scop: găsim α care minimizează $J(\theta)$ rapid și stabil



→ Ex. valori utile: 0.001, 0.003, 0.01, 0.03, 0.1, ...

În practică → Începem cu mai multe valori α și observăm cum scade costul

→ Când $J(\theta)$ devine aproape constant → converge



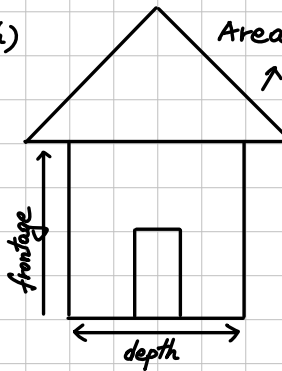
Alegerea feature-urilor

Exemplu: preț casă = $h(\text{frontage}, \text{depth})$

Putem crea un feature nou:
 $\text{area} = \text{frontage} \times \text{depth}$

Modelul devine:

$$f(x) = \theta_0 + \theta_1 \cdot \text{area}$$



$$\text{Area} = \text{frontage} \times \text{depth}$$

And we have a simple model:

$$\theta_0 + \theta_1 \cdot \text{Area}$$

POLYNOMIAL REGRESSION

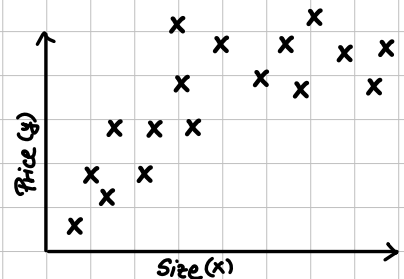
Când datele nu sunt liniare, adăugăm puteri:

$$x_1 = \text{size}$$

$$x_2 = \text{size}^2$$

$$x_3 = \text{size}^3$$

Rezultă modele quadratice sau cubice



NORMAL EQUATION

Putem calcula parametrii direct (analitic):

$$\theta = (X^T X)^{-1} X^T Y$$

Nu necesită GD

Bun pentru dataseturi mici, dar cost mare pt n mare

→ nr of features

Caracteristici	Gradient Descent	Normal Equation
Alege $\mathcal{L}$	✓ Necesită alegere $\mathcal{L}$	✗ Nu este necesar
Iterații	✓ Necesită multe iterații	✗ Nu necesită iterații
Performanță când n este mare	✓ Funcționează bine când n e foarte mare	✗ Devine lent când n este mare
Exemplu pt n mare	$n = 10^6$	$n \approx 1000$
Complexitate calcul	Iterativă, scalabilă	Operație pe matrice $n \times n$ complexitate $O(n^3)$

= Cheatsheet - Regresie Lineară Multiplă =

Modelul (ipoteza)

$$h_{\theta}(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_n x_n \rightarrow \text{în formă vectorială: } h_{\theta}(x) = \theta^T X$$

Forma matriceală:

$$\text{Pentru toate exemplele: } \hat{y} = \theta^T X$$

Funcția de cost:

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 = \frac{1}{2m} (X\theta - y)^T (X\theta - y)$$

Gradient Descent:

$$\theta := \theta - \mathcal{L} \frac{1}{m} X^T (X\theta - y)$$

Feature scaling:

$$x' = \frac{(x - \mu)}{\sigma}$$

Normal Equation:

$$\theta = (X^T X)^{-1} X^T Y$$

Metrici

$$RMSE = \sqrt{\frac{\sum (h_{\theta}(x^{(i)}) - y^{(i)})^2}{m}} \quad R^2 = 1 - \frac{\sum (y - \hat{y})^2}{\sum (y - \bar{y})^2}$$

Concept	Formula	Implementare Python
Predicție	$h_{\theta}(x) = \theta^T X$	<code>y_pred = X @ theta</code>
Cost	$J(\theta) = (1/2m)(X\theta - Y)^T (X\theta - Y)$	<code>np.dot(res, res) / 2m</code>
Gradient	$(1/m) \cdot X^T (X\theta - Y)$	<code>np.dot(X.T, residuals) / m</code>
Update	$\theta := \theta - \mathcal{L} \cdot \text{grad}$	<code>theta = alpha * grad</code>
Normal Equation	$(X^T X)^{-1} X^T Y$	<code>np.linalg.pinv(X.T @ X) @ X.T @ y</code>

= Regresie liniară multiplă - Deducție =

Funcția de ipoteză:

$$h_{\theta}(x) = \theta_0 + \theta_1 x_1 + \dots + \theta_n x_n, \text{ unde } \theta = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_n \end{bmatrix} \in \mathbb{R}^{(n+1) \times 1}$$

→ $n+1$ → pt că avem 1 bias + n coeficienți

→ x → un vector de coloană

$$X = \begin{bmatrix} 1 & x_1^{(1)} & x_2^{(1)} & \dots & x_n^{(1)} \\ 1 & x_1^{(2)} & x_2^{(2)} & \dots & x_n^{(2)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_1^{(m)} & x_2^{(m)} & \dots & x_n^{(m)} \end{bmatrix} \in \mathbb{R}^{m \times (n+1)}$$

$$y = X \cdot \theta$$

Diagram illustrating the dimensions of the vectors in the equation $y = X \cdot \theta$. The matrix X is labeled $m \times (n+1)$ and the vector θ is labeled $(n+1) \times 1$. The resulting vector y is labeled $m \times 1$.

$$\begin{aligned} J(\theta) &= (1/2m) \sum (h_{\theta}(x^{(i)}) - y^{(i)})^2 \\ &= (1/2m) \cdot (X\theta - y)^T (X\theta - y) \\ &= (1/2m) \cdot (X\theta - y)^T (X\theta - y) \end{aligned}$$

funcția de cost

$$\text{Pentru că } r^T r = [r_1 \ r_2 \ \dots \ r_m] \cdot \begin{bmatrix} r_1 \\ r_2 \\ \vdots \\ r_m \end{bmatrix} = \sum_{i=1}^m r_i^2$$

$$J(\theta) = \frac{1}{2m} \cdot (X\theta - y)^T (X\theta - y)$$

Vrem să calculăm derivata gradientului

$$\text{Notăm } r = X\theta - y \Rightarrow J(\theta) = \frac{1}{2m} r^T r$$

$$r^T r = (X\theta - y)^T (X\theta - y) = (X^T \theta - y^T) (X\theta - y) = \theta^T X^T X \theta - 2y^T X \theta + y^T y$$

$$r^T r = \theta^T X^T X \theta - 2y^T X \theta + y^T y$$

$$(r^T r)' = 2X^T X \theta - 2X^T y + 0$$

↓

Atenție, derivăm matrici!

$$J(\theta)' = \frac{1}{2m} \cdot 2 \cdot (X^T X \theta - X^T y) = \frac{1}{m} X^T (X\theta - y)$$

= Mastering Regression =

BIAS & VARIANCE

- Echilibru între complexitate și generalizare
- OBIECTIV → Minimizarea erorilor pe date noi (test)
- Ne ajută să prezicem genul de erori care apar

BIAS → Eroare sistemică

- Cât de DEPARTE sunt valorile prezise față de cele reale

VARIANCE → Sensibilitatea la schimbarea datelor

- Cât de MULT fluctuează predicțiile bazat pe seturi diferite de training

SCOP → Generalizare bună pe date năvăzute

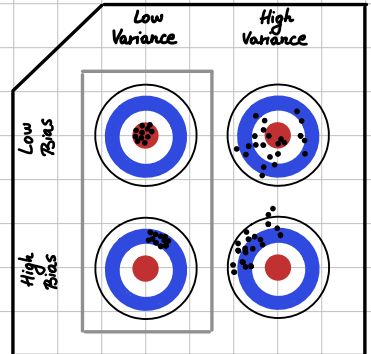
Modele prea simple → pierd patternuri → UNDERFITTING

Modele prea complexe → supraajustare → OVERFITTING

Alegerea arhitecturii corecte = Performanță Optimă

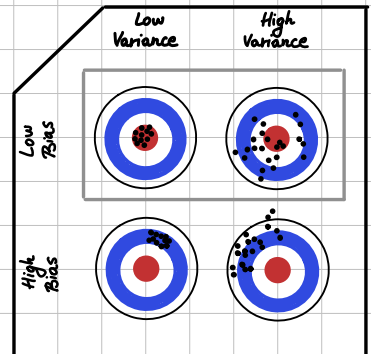
BIAS → Eroare Sistemică

- Diferența între predicții și valori reale
- Bias mare → Modelul nu surprinde relațiile reale
- Cauză → Model prea simplu



VARIANCE → Sensibilitate la date

- Fluctuația predicțiilor între seturi diferite de date
- Variance mare → Model instabil
- Cauză → Model prea complex



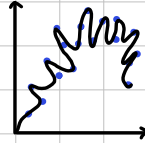
UNDERFITTING

- Bias Mare, Variance Mare
- Model prea simplu



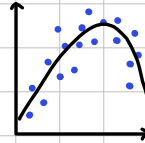
OVERFITTING

- Bias Mic, Variance Mare
- Model prea complex



FORMA OPTIMĂ

- Forma optimă surprinde patternul și generalizează pe date



INTUITIE

- Avem un grafic de puncte ca și cel alăturat
- Vrem să prezicem Y știind X
- Încercăm să fituim 3 modele de complexitate diferită:

- ① Model Liniar
- ② Model Polinomial
- ③ Model Polinomial de grad mare

Cum arată Bias/Variance?

○ Linear Regression

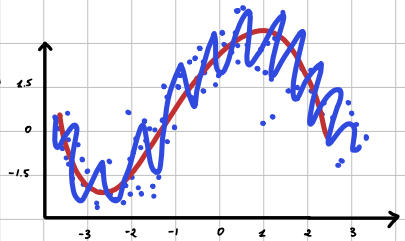
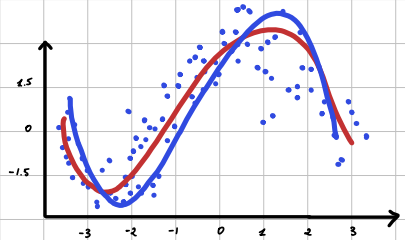
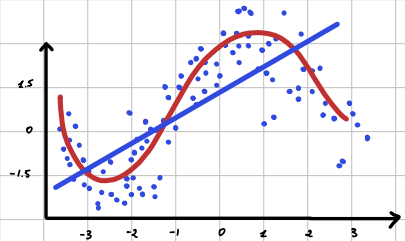
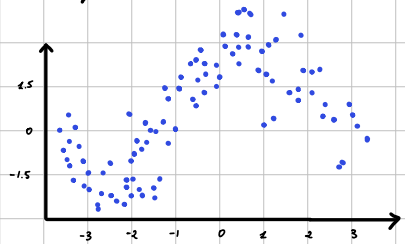
- Bias Mare
- Varianta Mică
- Underfitting

○ Regresie Polinomială (Grad 4)

- Bias Moderat
- Varianta Moderat

○ Regresie Polinomială (Grad 25)

- Bias Mic
- Varianta Mare
- Overfitting



Concluzie pentru Bias/Variance

→ Modelele cu BIAS mare:

- Sunt prea simpliste ca să captureze pattern-urile în date
- Provoacă underfitting
- Training/Testing accuracy overall nu e foarte bun

→ Modele cu VARIANCE mare:

- Sunt mult prea flexibile pe datele de training
- Provoacă overfitting
- Nu pot generaliza pe date pe care nu le-au mai văzut

Cum diagnosticăm?

- Plotăm erorile pe train și validare
 - Ambele mari → Bias Mare
 - Train mic, Validare mare → Variance Mare
- **CROSS-VALIDATION**
 - Împarte datele în subseturi
 - Antrenează/Testează pe combinații diferite
 - Estimează eroarea de generalizare
 - Detectează sensibilitatea la schimbarea datelor

REGULARIZARE

- Penalizează complexitatea modelului
- Adăugăm termen de penalizare în funcție de cost
- Previne overfitting-ul

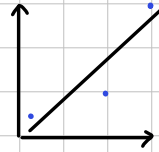
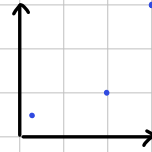
Exemplu: Luăm un dataset foarte simplu

$$f(x) = mx + n$$

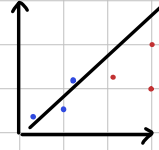
Vrem să minimizăm funcția de cost (MSE)

Folosim OLS

Rezultă OLS

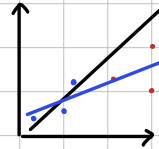


Dar dacă setul de test arată așa? →



RIDGE REGRESSION (L2)

$$f(x) = mx + n + \lambda m^2$$



LASSO REGRESSION (L1)

$$f(x) = mx + n + \lambda |m|$$

Reguli Rapide de Practică

Situație	Recomandare
Ai multe feature-uri dar puține exemple	$\mathcal{L}$ mai mare (regularizare puternică)
Ai multe date și puține feature-uri	$\mathcal{L}$ mai mic
Feature-urile sunt foarte corelate	Ridge (L2) funcționează mai bine
Vrei selecție automată de feature-uri	Lasso (L1) cu $\mathcal{L}$ mediu

NOTE → Ridge CV / Lasso CV din `sklearn.linear_model` ne pot da o idee de ce $\mathcal{L}$ să folosim

= METRICI =

MSE

MEAN SQUARED ERROR

$$MSE = \frac{1}{n} \sum (y_i - \hat{y}_i)^2$$

- Media pătratelor diferențelor dintre valorile reale și prezise
- Penalizează erori mari (pentru că le ridică la pătrat)
- 0 → Predicție perfectă
- Valori mari → Modelul greșește mult

Exemplu: Reale: [3, 5, 7], Predicții: [2, 6, 8] → MSE = 1

RMSE

ROOT MEAN SQUARE ERROR

$$RMSE = \sqrt{MSE}$$

- Rădăcina pătrată a MSE
- Are aceeași unitate ca ținta
- Interpretare ușoară → Eroarea medie reală
- Mai ușor de vizualizat (pentru oameni)

Exemplu: MSE = 236 → RMSE = $\sqrt{236} = 15.3$ → Modelul greșește cu aprox 15k €

R²

$$R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$$

- Arată cât de bine explică modelul variația din date
- Compară modelul cu o predicție bazată doar pe medie

Interpretare: $\begin{cases} R^2 = 1 \rightarrow \text{Perfect} \\ R^2 = 0 \rightarrow \text{Ca media} \\ R^2 = 0 \rightarrow \text{Mai prost ca media} \end{cases}$

Exemplu:

$R^2 = 0.4 \rightarrow$ Modelul explică 40% din variația reală a prețurilor

Concluzie și Comparatie

METRICA	CE MĂSOARĂ	INTERPRETARE	CÂND E UTILĂ
MSE	Media pătratelor erorilor	Penalizează tare erorile	Comparare între modele
RMSE	Rădăcina MSE	Eroarea medie "reală"	Interpretare umană
R ²	Cât de mult explică modelul	0-1 bun, <0 slab	Evaluare generală

= LOGISTIC REGRESSION & RECAP =

REGRESIE vs. CLASIFICARE

- REGRESIE → Variabila țintă numerică (ex: preț, temperatură)
- CLASIFICARE → Variabila țintă discretă (clase)

Tipuri de outputuri pentru clasificare:

- 1) BINARY : 0/1 (spam/not spam, malignă/benignă)
- 2) MULTI-CLASS : 0, 1, 2, ... (note, rating-uri)

LOGISTIC REGRESSION

- Deși numele conține "regression", este un clasificator
- SCOP → Prezice probabilitatea apariției unei clase (0 sau 1)
- Similar cu Linear Regression, dar folosește curba sigmoid pentru valori între [0, 1]
- Model simplu, popular și interpretabil

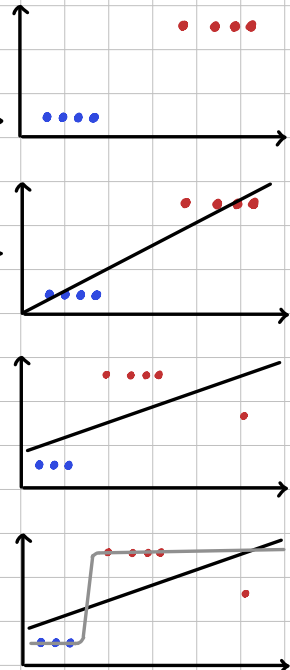
De la linear la logistic - intuiție

- Cum plotăm un target cu valoare discretă? →
• - benign ; • - malign

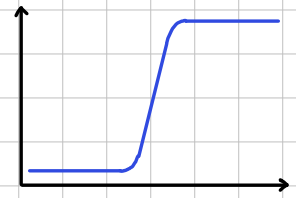
- Graficul reprezentat de linear regression →

- Dar dacă adăugăm un outlier, și rulăm din nou linear regression pe outlierul respectiv? →

- Logistic Regression pe datasetul cu outlieri →



SIGMOID $\rightarrow \sigma(z) = \frac{1}{1 + e^{-z}}$



Caracteristici:

- Domeniu real $\rightarrow$ interval $(0,1)$
- Netedă, derivabilă

În Logistic Regression: $z = \theta^T x$

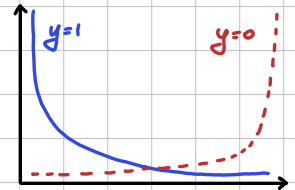
De la Probabilitate la Clasă:

- Modelul returnează o probabilitate $p \rightarrow p = \sigma(\theta^T x) = \frac{1}{1 + e^{-\theta^T x}}$
- În funcția de threshold, vrem ca probabilitatea să aparțină uneia dintre clase $\{0,1\}$
- Exemplu (threshold = 0.5)
 - Dacă $p \geq 0.5 \rightarrow$ Clasa 1
 - Altfel $\rightarrow$ Clasa 0

Când e potrivită?

- Tinta e binară (DA/NU)
- Datele sunt aproximativ linearly stable
- Vrei interpretabilitate (coeficienți arată impactul fiecărui feature)

LOG LOSS $\rightarrow L(p, y) = \frac{1}{n} \sum_i [-y_i \log(p) - (1 - y_i) \log(1 - p)]$



Interpretare:

- Dacă $y=1$ și p mare $\rightarrow$ Cost mic
- Dacă $y=0$ și p mare $\rightarrow$ Penalizare mare

Antrenarea Modelului (Gradient)

- Pași:
- ① Inițializare parametri
 - ② Calculează p
 - ③ Evaluează loss
 - ④ Actualizează $\text{param} := \text{param} - lr * \text{derivata}$
 - ⑤ Repetă până la convergență

Exemple de Aplicare:

- Email: spam / not spam
- Tranzacții Online: legitime / problematice
- Medicină: Tumoare malignă / benignă

Concluzii Logistic Regression binar

- Logistic Regression → model simplu, interpretabil
- Bun pentru clasificare binară
- LIMITE → Nu funcționează bine pe date foarte complexe sau non-lineare

Logistic Regression Multi-Class

- One vs Rest
 - Modelul trebuie să calculeze o probabilitate pe fiecare clasă
 - Exemplu - dacă avem 3 clase (A, B, C)
 - Modelul 1: A vs (B+C)
 - Modelul 2: B vs (A+C)
 - Modelul 3: C vs (A+B)
 - Fiecare produce o probabilitate p_A, p_B, p_C , iar modelul alege clasa cu probabilitatea cea mai mare
- Exemplu
 - $P(A) = 0.6$
 $P(B) = 0.4$
 $P(C) = 0.2 \rightarrow A$ câștigat
 - La nivel de cod, doar adăugăm
 $\rightarrow \text{LogisticRegression}(\text{multi_class} = \text{'auto'}) \text{ \#default}$

Logistic Regression Multi-Class

- **Softmax**

- În MULTINOMIAL LOGISTIC REGRESSION (SOFTMAX), vrem ca modelul să spună:

- "Care este probabilitatea ca exemplul x să aparțină fiecărei clase dintre mai multe?"

- Exemplu: Ai 3 clase $\rightarrow$ $\begin{cases} \text{floare A} \\ \text{floare B} \\ \text{floare C} \end{cases}$

- Modelul ar prezice:

$$\begin{array}{l|l} P(A) = 0.7 \\ P(B) = 0.2 \\ P(C) = 0.1 \end{array} \Rightarrow \text{Toate sumează } 1 \rightarrow \text{Adică modelul e sigur } 70\% \text{ că e floarea A}$$

SOFTMAX

$\rightarrow$

$$P(y=k|x) = \frac{e^{z_k}}{\sum_j e^{z_j}}, \text{ unde } z_k = m_k x + n_k$$

x - features

k - clasa

$P(y=k|x)$ - probabilitatea pt fiecare clasă k

Clasa	$z_k = m_k x + n_k$	e^{z_k}	Prob finală
A	2.0	7.39	$7.39 / (7.39 + 1.65 + 0.37) = 0.78$
B	0.5	1.6	0.17
C	-1.0	0.37	0.05

SOFTMAX vs OvR

SITUAȚIA	Alege OvR (One-vs-Rest)	Alege Softmax
Puține clase (ex: 3-5)	Da - simplu, rapid	Da - coerent și elegant
Multe clase (zeci/sute)	Da - mai eficient, parallel train	Mai lent
Clase independente (ex: poate fi și pisică și câine în același timp)	OvR (fiecare e o decizie separată)	Softmax fortează o singură clasă
Clase exclusive (ex: "sebosă", "versicolor", "virginica")	OvR poate da confuzii	Softmax e mai potrivit
Ai nevoie de probabilități coerente (care însumează 1)	Nu le garantează	Da, Softmax normalizează tot
Model interpretabil/Explicabil	Mai ușor de analizat (un model per clasă)	Dar puțin mai complex

Recap - introducere în ML

ML = folosirea datelor pentru a prezice comportamente, clase sau valori

Etapele unui model ML:

- Colectare și curățare date
- Explorare și selecție de feature-uri
- Alegerea și antrenarea algoritmului



- Evaluare și ajustare

= Evaluarea modelelor de clasificare =

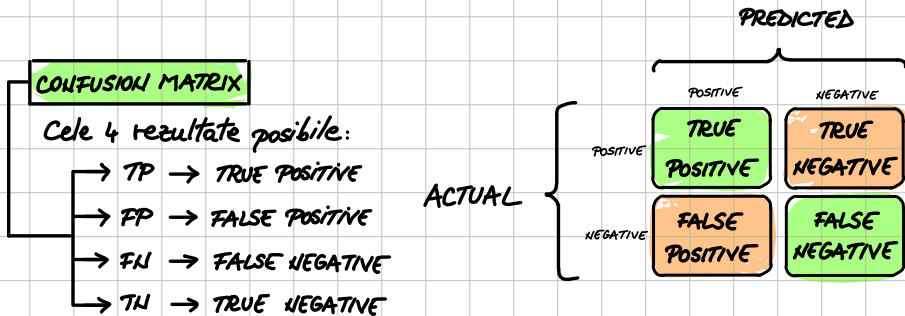
Mastering Regression

- Precizie, Recall
- Confusion Matrix
- Curba ROC

Cum știm dacă un model e bun?

- Un model nu e bun doar pentru că are acuratețe mare?
- Trebuie să știm unde greșește și ce tip de greșeli face

Punct de plecare: CONFUSION MATRIX



De ce folosim Confusion Matrix

- ✓ Ne arată ce tipuri de greșeli face modelul
- ✓ Ne ajută să decidem ce contează mai mult - PP sau FN
- ✓ Este baza tuturor modelelor de evaluare

PRECISION → $\text{Precision} = TP / (TP + FN)$

Exemplu: Modelul a prezis "pozitiv" de 100 ori,
88 corecte → $\text{Precision} = 88\%$

Un fel de a ne gândi la precision:

"De câte ori am avut dreptate când am spus că cineva e bolnav?"

ACTUAL

	PREDICTED	
	POSITIVE	NEGATIVE
ACTUAL	POSITIVE	TRUE POSITIVE TRUE NEGATIVE
	NEGATIVE	FALSE POSITIVE FALSE NEGATIVE

RECALL (SENSITIVITY) → $\text{Recall} = TP / (TP + FN)$

Exemplu: 200 de cazuri reale de bolnavi, modelul a detectat 180 → $\text{Recall} = 90\%$

Un fel de a ne gândi la recall:

"Dintre toți cei care CHIAR erau bolnavi, pe câți i-am găsit corect?"

ACTUAL

	PREDICTED	
	POSITIVE	NEGATIVE
ACTUAL	POSITIVE	TRUE POSITIVE FALSE POSITIVE
	NEGATIVE	TRUE NEGATIVE FALSE NEGATIVE

Echilibru între Precision și Recall

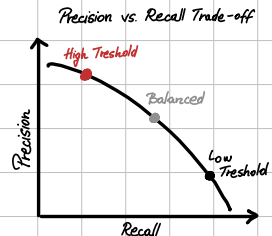
- Când creștem Recall-ul de obicei scade Precision-ul

↳ Un prag mai mic

→ Mai multe cazuri detectate (Recall ↑),
dar și mai multe greșeli (Precision ↓)

↗ Un prag mai mare

→ Mai puține alarme false (Precision ↑),
dar riscăm să ratăm cazuri reale (Recall ↓)



- Alegem echilibrul potrivit în funcție de scopul modelului

SPECIFICITY → $\text{Specificity} = \frac{TN}{TN+FP}$

Exemplu: 100 persoane sănătoase, modelul a detectat 80 → $\text{Specificity} = 80\%$.

Un fel de a ne gândi la specificity:
 "Dintre toți cei care CHIAR erau sănătoși, pe câți i-am găsit corect?"

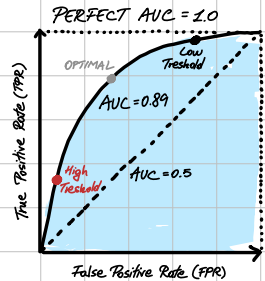
ACTUAL

		PREDICTED	
		POSITIVE	NEGATIVE
ACTUAL	POSITIVE	TRUE POSITIVE	FALSE POSITIVE
	NEGATIVE	FALSE NEGATIVE	TRUE NEGATIVE

ROC Curve

→ Arată cât de bine separă modelul cazurile pozitive de cele negative:

- Fiecare punct = un prag de decizie diferit
- Cu cât e mai sus și spre stânga → model mai bun
- Linia diagonală = model care ghicește la întâmplare



ROC vs PRECISION/RECALL

Vrei să clasifici dacă e sau NU o pisică în imagine. Setul de date arată așa:

- 990 poze FĂRĂ pisici - set 0
- 10 CU pisici - set 1

Faci clasificarea, și modelul scoate:

$$\begin{aligned} \text{FPR} &= 1 - \text{SPECIFICITY} \\ &= 1 - \frac{TN}{TN+FP} \\ &= 0.05 = 5\% \end{aligned}$$

$$\begin{aligned} \text{PRECISION} &= \frac{TP}{TP+FP} \\ &= 66.7\% \end{aligned}$$

Matrice de confuzie (pisică=1)

		15 detectate	
Etichetă reală	fără pisică (0)	985	5
	pisică (1)	0	10
		fără pisică (0)	pisică (1)
		Etichetă prezisă	

Când o folosim pe fiecare?

SITUAȚIE	Folosește ROC Curve	Folosește Precision-Recall Curve
Clase echilibrate (pozitive ≈ negative)	✓ DA	X Nu e necesar
Clase dezechilibrate (pozitive << negative)	X NU	✓ Da
Vrei să vezi cât de bine separă modelul clasele	✓ ROC e ideal	✓ Poți folosi și PR, dar ROC e mai intuitiv
Vrei să vezi calitatea predicțiilor pozitive	X Nu prea relevant	✓ PR e perfect
Exemple: Diagnostic medical echilibrat (ex: benign/malign)	✓ ROC	X PR
Exemple: fraude, spam, defecte rare	X ROC	✓ PR

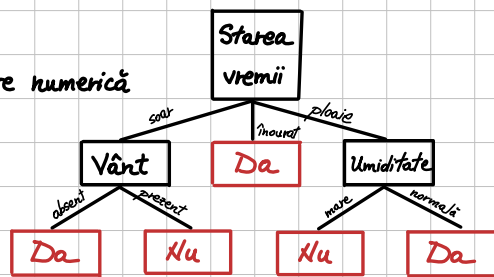
= DECISION TREE & RANDOM FOREST =

Ce este un Decision Tree?

- Un instrument pentru luarea deciziilor automate
- Se bazează pe întrebări: dacă o condiție e:
 - adevărată → merge la stânga
 - falsă → merge la dreapta
- Rezultatul final = o decizie sau o valoare numerică

Cum gândesc arborii

- NOD = o întrebare
- RAMURĂ = Un răspuns (True/False)
- FRUNZĂ = O decizie finală
- Începem de sus (ROOT) și coborâm până la o FRUNZĂ

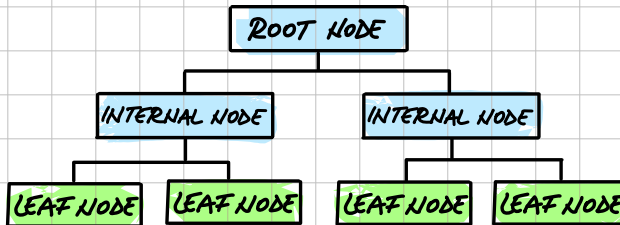


Clasificare vs Regresie

- Classification Tree → prezice o etichetă (Bolnav/Sănătos)
- Regression Tree → prezice o valoare numerică (Prețul unei case)
- Ambele împart datele pas cu pas

Structura unui arbore

- **ROOT NODE** → Punctul de start
- **BRANCHES** → Răspunsurile la întrebări
- **INTERNAL NODES** → Conțin condițiile
- **LEAVES** → Rezultatele finale



Cum se construiește un arbore

- ① Pornim cu datele și featurele
- ② Alegem featureul care separă cel mai bine clasele
- ③ Repetăm până ajungem la frunze pure
- ④ O frunză pură = toate exemplele dintr-o singură clasă

Vârsta	Venit	Cumpărat?
tânăr	mic	NU
tânăr	mare	DA
adult	mare	DA
bătrân	mic	NU

Ce înseamnă impuritate?

- Un nod "pur" = toate exemplele sunt la fel
- Un nod "impur" = amestec de clase (DA/NU)

Măsuri folosite:

- Gini Impurity
- Entropie

GINI IMPURITY → $Gini = 1 - P(DA)^2 - P(NU)^2$

- 0 → Nod Complet Pur
- Valoare mare → Amestec de clase

Exemple: Avem 3 "DA" și 1 "NU" (valori țintă raportate la același feature)

Formula: $1 - (3/4)^2 - (1/4)^2 = 0.375$

ENTROPY & INFORMATION GAIN

ENTROPY

$$H = -P(DA) \log_2(P(DA)) - P(NU) \log_2(P(NU))$$

→ Cât de amestecate sunt datele

INFORMATION GAIN

$$\text{Entropy(before)} - \text{Entropy(after)}$$

→ Câtă claritate câștigăm după split

→ Alegem feature-ul cu cel mai mare INFORMATION GAIN

Cum alegem primul feature

- Testăm fiecare feature și calculăm impuritatea totală
- Alegem featureul cu impuritatea cea mai mică
- Repetăm procesul pentru fiecare NODE

Calculăm GINI IMPURITY pentru "Venit"

- Venit = mic → {NU, NU}
- Venit = mare → {DA, DA}

$$\text{GINI}(\text{mic}) = 1 - (0)^2 - (1)^2 = 0$$

$$\text{GINI}(\text{mare}) = 1 - (1)^2 - (0)^2 = 0$$

Vârsta	Venit	Cumpărat?
tânăr	mic	NU
tânăr	mare	DA
adult	mare	DA
bătrân	mic	NU

→ GINI total pentru "Venit" = 0 → Set perfect curat

Calculăm GINI IMPURITY pentru "Vârsta"

- tânăr → {NU, DA}
- adult → {DA}
- bătrân → {NU}

$$\text{GINI}(\text{tânăr}) = 1 - (0.5)^2 - (0.5)^2 = 0.5$$

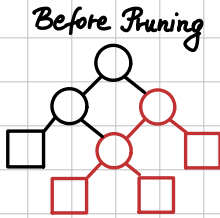
$$\text{GINI}(\text{adult}) = 0$$

$$\text{GINI}(\text{bătrân}) = 0$$

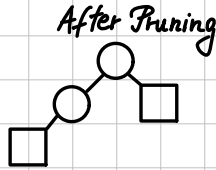
$$\begin{aligned} \text{GINI total pe "Vârsta"} \\ = \left(\frac{2}{4}\right) \cdot 0.5 + \left(\frac{1}{4}\right) \cdot 0 + \left(\frac{1}{4}\right) \cdot 0 = 0.25 \end{aligned}$$

PRUNING → Tăierea arborelui

- Arborele poate deveni prea complex (overfitting)
- PRUNING = eliminăm ramurile inutile
- Arborele devine mai scurt și mai interpretabil
- Reduce overfitting și crește acurătatea



Eliminating
unnecessary
branches



- Tree becomes shorter and more interpretable
- Reduces overfitting and increases accuracy

✓ Avantaje

- Ușor de folosit
- Funcționează cu date numerice și categorice

x Dezavantaje

- Instabil la schimbări mici în date
- Poate supraînvața (overfit)

REGRESSION TREE

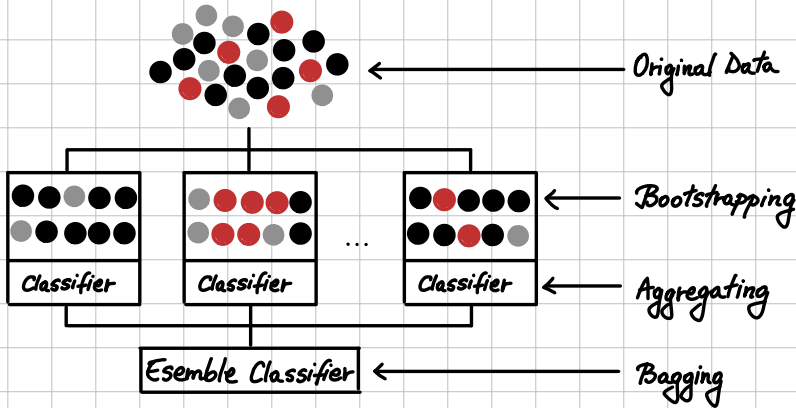
- Arborele care prezice valori numerice
- În loc de "DA/NU", fiecare frunză conține o medie
- Fiecare split reduce variația

RANDOM FOREST

- Combinații de mai mulți arbori (esemble)
- Fiecare primește un bootstrap (aleator, cu înlocuire)
- Fiecare folosește un subset random de features
- Predicție finală = votul majoritar

BOOTSTRAPPING → Antrenăm fiecare arbore pe un subset random

- Setul original → Mai multe subseturi → arbori → agregare
- Beneficii:
 - Diversitate între arbori
 - Reduce overfitting
 - Estimare erori



BAGGING → Combină predicțiile prin vot / medie

OUT-OF-BAG ERROR

- Datele nefolosite la antrenare pot fi testate pe acel arbore
- Media erorilor dă OUT-OF-BAG ACCURACY
- Este o metodă de validare internă

Când folosim decision tree?

- Machine Learning: Clasificare, Regresie
- Medicină: diagnostic, decizii clinice
- Business: segmentarea clienților, predicții de vânțări
- Finanțe: evaluarea riscului, detectarea fraudelor
- Juridic: suport pentru decizii, clasificarea documentelor
- Datele au relații non-liniare între caracteristici și etichetă
- Ai caracteristici categorice sau mixte (numerice + categorice)
- Vrei un model interpretabil vizual (ușor de explicat unui stakeholder)
- Ai nevoie de un model care să gestioneze datele lipsă sau să nu necesite scalare

ACCURACY vs F1-SCORE

ACCURACY

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

→ Proportia de predicții corecte din totalul cazurilor

F1-SCORE

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

→ Media armonică între Precision și Recall

MACRO vs WEIGHTED AVG

MACRO AVG

→ Media simplă a scorurilor pe toate clasele

- Fiecare clasă contează în mod egal, indiferent de mărime
- Bun pentru seturi dezechilibrate - vezi performanța reală pe clase mici

WEIGHTED AVG

→ Media ponderată cu support (nr de exemple din fiecare clasă)

- Clasele mari au greutate mai mare în calcul
- Poate ascunde performanță slabă pe clasele mici

Exemplu:

Clasă	Nr. exemplu	F1-Score
0	95	1.0
1	5	0.5

$$\text{Macro Avg} = (1.0 + 0.5) / 2 = 0.75$$

$$\text{Weighted Avg} = (1.0 \cdot 9.5 + 0.5 \cdot 5) / 100 = 0.975$$

= CONFUSION MATRIX & GRADIENT BOOSTING =

GRADIENT BOOSTING

→ TPs / FNs / FPs / TNs

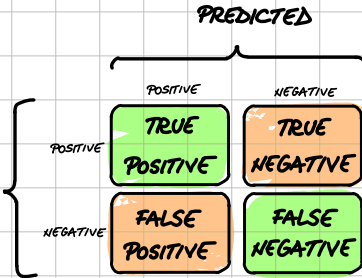
→ Gradient Boosting Algorithm

CONFUSION MATRIX

Cele 4 rezultate posibile:

- TP → TRUE POSITIVE
- FP → FALSE POSITIVE
- FN → FALSE NEGATIVE
- TN → TRUE NEGATIVE

ACTUAL



TERMEN	EXPLICAȚIE	EXEMPLU (SPAM vs Non-SPAM)
TP (True Positive)	Modelul prezice POSITIV și este CORECT	Email spam → Prezis Spam
FP (False Positive)	Modelul prezice POSITIV, dar este GREȘIT	Email normal → Prezis Spam
FN (False Negative)	Modelul prezice NEGATIV, dar este GREȘIT	Email spam → Prezis normal
TN (True Negative)	Modelul prezice NEGATIV și este CORECT	Email normal → Prezis normal

Exemplu: Gradient Boosting - date de pornire

Scop: Învățăm să folosim o funcție $F(x)$ care să prezică y din x

i	x	y (target)	Predicție inițială $F_0(x)$	Reziduu $r^{(1)} = y - F_0(x)$
1	1	3	3.5	$3 - 3.5 = -0.5$
2	2	2	3.5	$2 - 3.5 = -1.5$
3	3	4	3.5	$4 - 3.5 = +0.5$
4	4	5	3.5	$5 - 3.5 = +1.5$

La Gradient Boosting cu MSE, modelul inițial este media valorilor lui y :

$$F_0(x) = \bar{y} = \frac{3+2+4+6}{4} = \frac{14}{4} = 3.5$$

GRADIENT BOOSTING - STEPS

i	x	y (target)	Predicție inițială $F_0(x)$	Rezidu $r^{(1)} = y - F_0(x)$
1	1	3	3.5	$3 - 3.5 = -0.5$
2	2	2	3.5	$2 - 3.5 = -1.5$
3	3	4	3.5	$4 - 3.5 = +0.5$
4	4	5	3.5	$5 - 3.5 = +1.5$

① INITIALIZARE

- $F_0 =$ media lui y
- Reziduuri: $R = y - F_0$

② Arbore pentru reziduuri (STUMP)

- Ordonezi punctele după x
- Split-uri posibile = mijlocul între valorile lui x
- Pentru fiecare split:
 - Împarți datele în A și B
 - Calculezi media reziduurilor în A și B
 - Calculezi eroarea totală: $(r - \mu_A)^2 + (r - \mu_B)^2$
- Alegi split-ul cu eroarea totală minimă
- Arborele prezice μ_A și μ_B

③ Update model

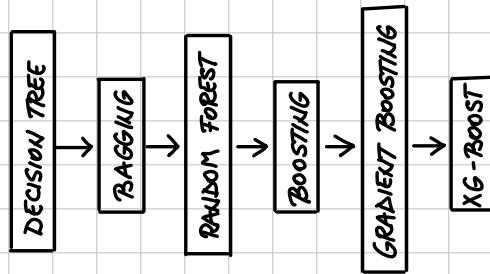
- $F_m = F_{m-1} + v \cdot h_m$
- Reziduuri noi: $r = y - F_m$

④ Repeat

- Repeti construcția de arbori pe noile reziduuri

= XGBOOST =

Evolution of Tree Algorithm



De ce XGBoost (Extrem Gradient Boost)?

- Creat pentru dataseturi complexe, mari și sparse
- Optim pentru:
 - date cu multe features
 - lipsă de valori
 - relații non-lineare
- Unul dintre cei mai folosiți algoritmi în industrie (competiții Kaggle, modelling real-world)
- În notebook vom folosi un dataset mic doar pentru simplitate

XGBoost = Gradient Boosting + Optimizări

XGBoost = Gradient Boosting (conceptul)

O multime de optimizări de performanță:

- Regularization (L1, L2)
- A unique Regression Tree model
- Approximate Greedy Algorithm
- Weighted Quantile Sketch
- Sparsity-Aware Split Finding
- Parallel Learning
- Cache-Aware Access Pattern
- Out-Of-Core computation (cu blocuri)

Practic: Boosting obișnuit, dar mai rapid, mai stabil, mai bun

În Gradient Boosting

- Pornim cu o predicție inițială (constantă)
- Calculăm residuals = reals - predicție
- Învățăm un arbore care prezice aceste residuals
- Adăugăm arborele cu un mic learning rate
- Repetăm procesul

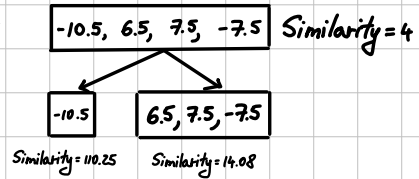
XGBoost folosește idee, dar îmbunătățită

XGBoost folosește un singur tip special de arbore

XGBoost Tree - un Tree optimizat

- Toate punctele încep într-o singură frunză
- Se calculează un similarity score:

$$\text{Similarity Score} = \frac{\text{Sum of Residuals Squared}}{\text{Ntr of Residuals} + \lambda}$$



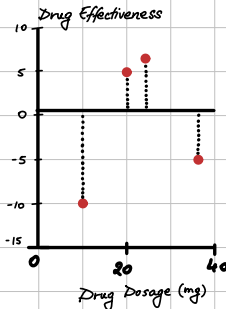
λ = regularizare (reduce overfitting)

Întrebare: Dacă împărțim frunza (leaf) în 2, ne crește calitatea?

Intuitie pentru Similarity Score

Ce ne spune Similarity Score?

- Dacă residuals sunt foarte diferite
→ Se "anulează" → Scor Mic
- Dacă residuals sunt asemănătoare
→ Se adună → Scor Mare



Un scor mare înseamnă că frunza este "curată" și arborele învață ceva real

Cum evaluăm un split? (GAIN)

Pentru un posibil split:

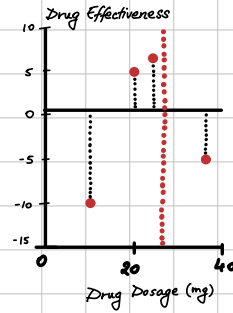
$$\text{Gain} = \text{Left}_{\text{SIMILARITY}} + \text{Right}_{\text{SIMILARITY}} - \text{Root}_{\text{SIMILARITY}}$$

- Dacă $\text{GAIN} > 0$, split-ul ajută
- Dacă GAIN e mic sau negativ → nu merită

Cum se caută cel mai bun split?

- ① Sortăm după feature
- ② Testăm posibile threshold-uri
- ③ Calculăm gain pentru fiecare
- ④ Alegem splitul cu gain maxim
- ⑤ Repetăm pentru fiecare frunză

În practică, XGBoost optimizează masiv pași 1-3



REGULARIZARE

$$\text{Similarity Score} = \frac{\text{Sum of Residuals Squared}}{\text{Nr of Residuals} + \lambda}$$

- λ (lambda)
- Regularizare L2

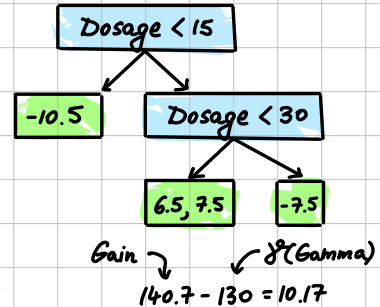
→ Penalizează frunzele cu puține observații

PRUNING

- γ (Gamma)
- Control pentru pruning
- Dacă $\text{Gain} - \text{Gamma} < 0 \rightarrow$ Eliminăm ramura

→ γ = Reduce sensibilitatea → Menține arbori simpli

→ γ este numit `min_split_loss` în implementarea XGBoost



ETA

LEARNING RATE

XGBoost numește learning rate ETA

Valoarea default: 0.3

Cu cât ETA e mai mic, cu atât avem:

- Boosting mai lent
- Mai mulți arbori
- Modele mai stabile și mai puțin over-fit

XGBoost pentru Clasificare

- Este identic ca logică cu regresia
- Diferența este în formula cu similitudine, care folosește:

$$\frac{(\sum \text{Residual}_i)}{(\sum [\text{Previous Probability}_i \times (1 - \text{Previous Probability}_i)] + \lambda)}$$

Approximate Greedy Algorithm

De ce nu testăm toate thresholdurile?

Pentru că ar fi foarte lent:

- Multe features
- Multe combinații
- Enorme costuri de calcul

Soluție → Greedy + Approximations, suficient de bune și foarte rapide

Weighted Quantile Sketch

Cum alegem thresholdurile fără să le testăm pe toate?

- Împărțim datele în quantile
- Testăm doar quantilele cu threshold-uri
- Foarte rapid
- Weighted = contează mult quantilele care au grad mare

Este fundația vitezei XGBoost

Sparsity-Aware Split Finding

Ce facem cu valorile lipsă (missing values)?

XGBoost:

- Încearcă split cu missing în stânga
- Încearcă split cu missing în dreapta
- Compară Gains
- Alege varianta optimă

Nu pierdem timpul imputând date

Parallel Learning

- Fiecare feature poate fi procesat într-un thread separat
- XGBoost face calculul în paralel în loc de secvențial
- Mult mai rapid decât gradient boosting standard

Cache-aware & Out-Of-Core

- Organizează date în blocuri pentru citire eficientă
- Folosește cache-aware access pentru utilizarea optimă a memoriei
- Poate antrena direct pe disk (out-of-core)

De ce e XGBoost atât de bun în practică?

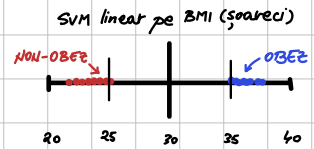
- Rulează rapid
- Face față dataseturilor mari
- Gestionare excelentă a missing values
- Regularizare = modele stabile
- Funcționează exceptional pe probleme reale
- Ușor de interpretat și tune-wit
- Se integrează în pipeline-uri industriale

= Support Vector Machine (SVM) =

Problema inițială

- Clasificăm persoane obeze vs. non-obeze
- Avem observații (ex: greutate, BMI, etc.)
- Alegem un threshold care separă cele 2 clase

Punctele noi se clasifică în funcție de poziția față de threshold



Problema threshold-ului

Ce se întâmplă dacă un punct e exact pe linia de separare?

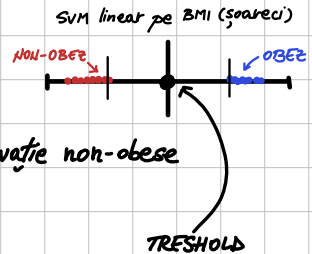
- Dacă e peste threshold → obez
- Dacă e sub threshold → non-obeze (greutate normală)
- Dar threshold-ul poate fi instabil

Poate fi prea aproape de o clasă → Clasificări greșite

Îmbunătățirea thresholdului

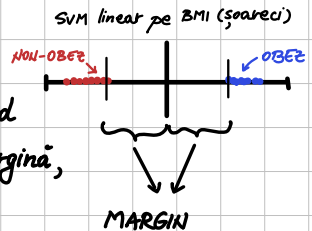
Putem recalcula threshold-ul:

- Luăm ultima observație obeze și prima observație non-obeze
- Setăm threshold-ul la mijloc
- Astfel evităm clasificările ambigue



Ce este MARGIN?

- Distanța dintre "observații de margine" și threshold
- Dacă alegem threshold-ul cu cea mai mare margină, obținem: Maximal Margin Classifier

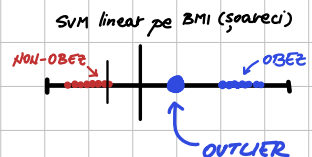


Problema Maximal Margin Classifier

Sensibil la outliers

Un singur outlier poate:

- strică thresholdul
- micșora marginea
- produce un model prost pe date noi



SOFT MARGIN

Acceptăm intenționat unele misclasificări:

- Nu luăm ultimul și primul punct

Ne uităm la:

- Penultimul / Al 2-lea punct
- Antepenultimul / Al 3-lea punct

Creăm un soft margin classifier

NOTE: Parametrul din cod care ne ajută cu asta este C

Bias-Variance Tradeoff

De ce funcționează SOFT MARGIN?

MAX MARGIN → LOW BIAS / HIGH VARIANCE

→ Bun pe training, slab pe test

SOFT MARGIN → HIGH BIAS / LOW VARIANCE

Cum alegem cea mai bună margine?

Folosim CROSS VALIDATION

- Testăm diferite valori pentru margine
- Alegem varianta care performează cel mai bine pe date nevăzute

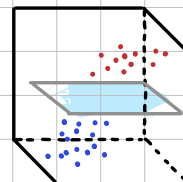
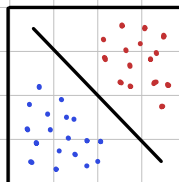
SUPPORT VECTOR CLASSIFIER

Când alegem SOFT MARGIN + THRESHOLD OPTIM = SUPPORT VECTOR CLASSIFIER
Punctele care definesc marginea = Support Vectors

În 2D → linie

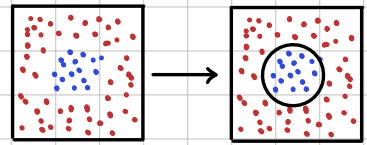
În 3D → plan

În 4D → hyperplan



Când datele se suprapun?

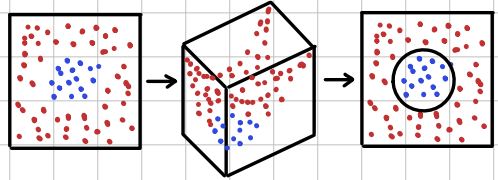
- Un linear classifier nu funcționează
- Uneori relația este non-liniară
- Avem nevoie să mutăm datele într-o dimensiune mai mare



De ce ridicăm la pătrat?

Transformare non-liniară:

- Date 1D → Transformare în 2D
- Putem desena un separator liniar în 2D
- Ideea SVM:
 - Pornim din dimensiune mică
 - Proiectăm în dimensiune mare
 - Aplicăm un Support Vector Classifier



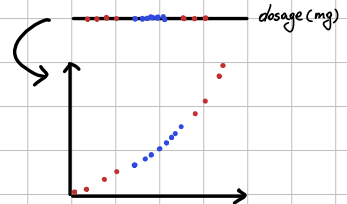
KERNEL FUNCTIONS

Cum ridicăm datele în dimensiuni mari?

- SVM folosim KERNEL FUNCTIONS
- Ele calculează relații între puncte, fără transformare explicită → KERNEL TRICK
- Putem obține separare liniară în spații mari fără cost calcul mare

POLYNOMIAL KERNEL

- Are un parametru degree (d)
- Degree = 1, 2, 3, ...
- Controlăm cât de complexă devine separare
- Alegem d prin cross-validation



RADIAL (RBF) KERNEL

- Mapează datele într-un spațiu infinit-dimensional
- Puncte apropiate → influență mare
- Puncte îndepărtate → influență mică
- Ideal când datele sunt "încălcite" și greu de separat

KERNAL TRICK

- Kernelul nu generează efectiv dimensiuni noi
- Doar calculează similaritatea dintre puncte
- Modelul lucrează ca și cum ar fi în spațiu mare → fără cost

Când folosim SVM?

SVM e bun când:

- Avem puține mostre ($< 20k$)
- Avem multe features (high-dimensional)
- Datele sunt curate, scalate
- Problema are formă complexă dar nu prea mare
- Vrem regularizare bună și un classifier stabil

Cel mai potrivit în:

- NLP
- Date biomedicale
- Date high-dimensional

Exemple unde SVM câștigă

- Clasificare text → lipsă de date, multe features
- Biologie moleculară pe gene → 100 probe / 10000 features
- Orice dataset mic dar cu feature engineering intens

Support Vector Machines - Rezumat

- Max Margin classifier
- Soft Margin pentru robustețe
- Kernel Trick pentru non-linearitate
- Bun pentru dataseturi mici / medii cu multe features